

Lab 14 — Wireless, Low Power: BLE GATT with NimBLE

ESP32-S3 FIRMWARE LAB SERIES · PHASE 2 · PAST THE COURSEWORK

Estimated time	3–4 hrs
Introduces	the GATT data model, notifications, UUIDs
Prerequisites	Labs 2 and 7. No Bluetooth background assumed.
Beyond the board	A phone with a BLE scanner app such as nRF Connect.
Target	ESP32-S3 (written against an N16R8 module: 16 MB flash, 8 MB octal PSRAM)
Framework	ESP-IDF v5.5, CMake and Ninja, driven by <code>idf.py</code>

NOTE

How to read this lab

Sections run **§14.1 objective**, **§14.2 parts and wiring**, **§14.3 how it works**, then the code, a line-by-line walkthrough, the documentation map, and verification. Callout boxes are tagged **[WARNING]**, **[KEY POINT]**, **[NOTE]**, **[DOCS]** and **[TM4C BRIDGE]**. Figures are given as captioned text and tables so every value in them is searchable and checkable.

The **TM4C bridge** notes map each topic onto the equivalent bare-metal, register-level work on a TI TM4C123 (Cortex-M4F). Skip them if that is not your background; nothing later depends on them.

14.1 Objective & TM4C bridge

Expose your sensor data over Bluetooth Low Energy so a phone can read it and subscribe to live updates. BLE is the connectivity option for battery devices, and it is the one most likely to appear in a wearable or consumer-product interview.

TM4C BRIDGE

None. This is application-layer wireless messaging, several levels above anything bare metal, and it introduces a genuinely new way of thinking about data — you do not send messages, you *publish a structured table of values* and let clients read or subscribe.

14.2 Parts & wiring

Item	Status	Notes
ESP32-S3 board	BLE radio is on the module	Bluetooth 5 (LE). No classic Bluetooth on the S3.

Item	Status	Notes
A phone with a BLE scanner app	Required	nRF Connect (Nordic) is the standard tool and is free on both platforms. It is effectively a debugger for BLE and you should install it before starting.

No wiring. Note that BLE and Wi-Fi share one radio, so running both simultaneously is possible but they contend; if you combine Labs 13 and 14 later, expect to think about that.

14.3 How it works

BLE has a genuinely different shape from every other interface in this series, and the shape is the hard part. Assume no Bluetooth background.

Bluetooth Low Energy is not Bluetooth

Despite the name, **BLE and classic Bluetooth are different protocols** that happen to share a brand and a frequency band. Classic Bluetooth is designed for continuous streams — audio to headphones, a file transfer — and keeps a connection alive constantly. BLE is designed for devices that say something small, occasionally, and then shut the radio off for as long as possible.

That difference in purpose explains everything about how BLE is built. It is not a slower Bluetooth; it is a protocol optimised around the assumption that *the radio being on is the expensive thing*. The ESP32-S3 supports **only BLE**, not classic — the original ESP32 supported both.

The idea that takes the longest to accept: BLE is a database, not a pipe

UART, SPI and Wi-Fi are all fundamentally about *sending messages*. You have bytes, you push them, they arrive. Almost everyone approaches BLE looking for the "send" function, and there isn't one in the sense you expect.

A BLE peripheral instead **publishes a small structured table of values**, and clients read from it, write to it, or ask to be told when a value changes. That table and the rules around it are called **GATT** — the Generic Attribute Profile. Your firmware's job is to *declare* the table and keep the values current; the stack handles everything about who reads what and when.

The closest familiar analogy is a tiny web API rather than a serial port. You are not writing to a stream; you are exposing resources that a client can query.

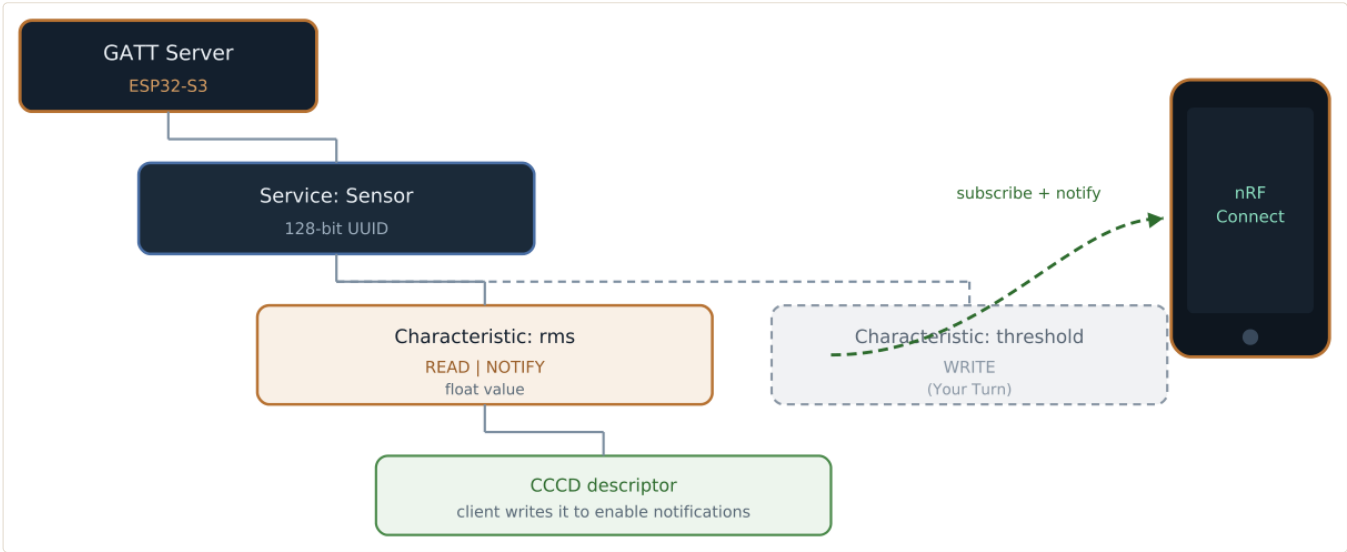


Figure 14.1. The GATT tree. A peripheral publishes services; each service holds characteristics; each characteristic is one value with properties saying whether it can be read, written or subscribed to. The CCCD descriptor is where a client records its subscription.

BLE GATT tree as exposed by the peripheral.

Hierarchy: - GATT Server (the ESP32-S3) - Service: Vibration (128-bit UUID) - Characteristic: rms, properties READ and NOTIFY, value is a 4-byte float - CCCD descriptor: the client writes 1 here to enable notifications - Characteristic: threshold, property WRITE, value a float (shown dashed as the "your turn" extension)

A phone running nRF Connect connects, discovers this tree, and subscribes, after which notifications flow from the board to the phone.

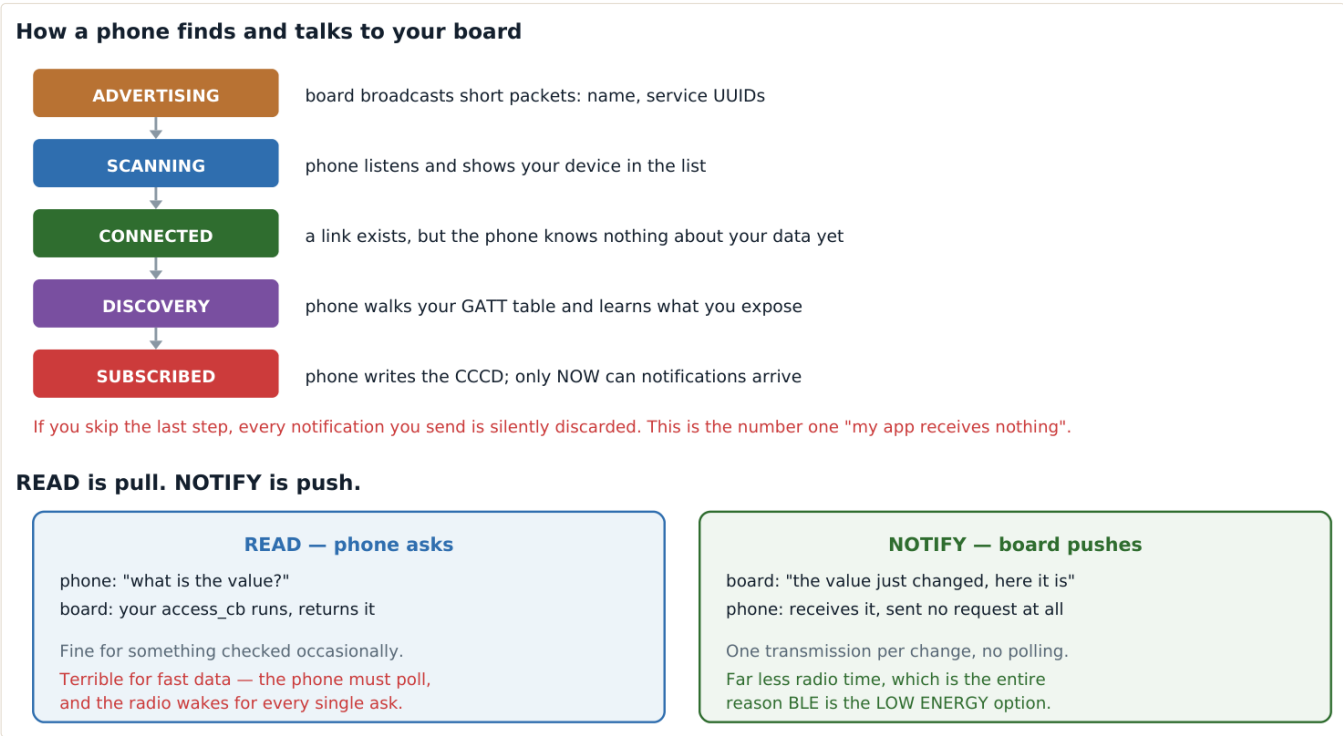
The three levels of the tree

Level	What it is	Example
Service	A logical grouping of related values	"Battery Service", or your own "Vibration Service"
Characteristic	One individual value, with properties	Battery percentage; your RMS figure
Descriptor	Metadata attached to a characteristic	The CCCD , which is how a client switches notifications on

Everything is named by a **UUID**, a 128-bit identifier. The Bluetooth SIG has adopted short 16-bit UUIDs for standard services — heart rate, battery level, device information — so that any phone app can recognise them without prior knowledge. Anything you invent uses a full 128-bit UUID, which is why your custom service's identifier is a long hex string: with 128 bits, you can generate one at random and be confident nobody else on earth has picked the same one.

Prefer a standard UUID when one fits. If you are exposing a battery level, use the adopted Battery Service and every generic BLE app will display it correctly with no work from you.

How a phone actually finds you



NOTIFY is push: the board says "the value just changed, here it is" and the phone receives it having sent no request at all. One transmission per change, no polling. Far less radio time, which is the entire reason BLE is the LOW ENERGY option.

Five stages, and knowing them turns "it doesn't work" into a specific question about which stage failed.

Advertising. Your board periodically broadcasts a short packet containing its name and the UUIDs of services it offers. It is shouting into the room; nobody has to be listening. This costs almost no energy because the packets are tiny and infrequent.

Scanning. The phone listens for those broadcasts and builds a list. This is what populates the device list in nRF Connect.

Connecting. The phone establishes a link. At this point you have a connection but the phone still knows nothing about what data you offer.

Discovery. The phone walks your GATT table and learns what services and characteristics exist. This is why the tree has to be *declared* rather than just used — the client discovers it at runtime.

Subscribing. The phone writes to the CCCD of a characteristic to say "tell me when this changes."

Read is pull, notify is push — and why that is the whole point

A **read** is client-initiated: the phone asks, your access callback runs, you return the current value. That is fine for something a user checks occasionally.

It is terrible for a value that changes constantly, because the phone would have to poll — and every poll means waking the radio, exchanging packets, and burning energy on both devices to often learn nothing has changed.

A **notification** is peripheral-initiated: the instant your value changes, you push it, with no request. One transmission per actual change. **That is the mechanism that makes BLE low-energy**, far more than any clever modulation: the design lets both radios stay off almost all the time.

WARNING

A peripheral cannot force data onto a client

Notifications only reach the phone if it has **subscribed** by writing to the CCCD — in nRF Connect, tapping the subscribe icon. Until then, every notification you send is silently discarded by the stack, with no error returned to your code.

"My app receives nothing" is nearly always this, not a firmware bug. Check the subscription before you check anything else, and it will save you an evening.

There is also **indication**, a variant that requires the client to acknowledge each message. It is slower but confirms delivery, and you would use it for something that must not be missed.

Why NimBLE

ESP-IDF ships two BLE host stacks. **Bluedroid** is the older one and also supports classic Bluetooth, which the S3 cannot use anyway. **NimBLE** is BLE-only and substantially smaller in both flash and RAM — on the order of tens of kilobytes saved, which is real on a part with 512 KB of SRAM.

For a BLE-only device NimBLE is the sensible default, and it is what this lab uses. Being able to say *why* you chose a stack is worth more in an interview than knowing either API.

What this costs you in memory

Worth knowing before you are surprised by it: enabling BLE consumes a substantial chunk of flash for the stack and a meaningful slice of RAM for its buffers and connection state. On a project already using Wi-Fi, PSRAM buffers and a display, this is the point where you start actually reading `idf.py size` output rather than assuming things fit.

14.4 Procedure & code

The stack initialisation is long and largely boilerplate; start from the `bleprph` example in your ESP-IDF installation rather than typing it. The parts that are *yours* are the GATT table and the notify call, so those are what appear here.

```
static const struct ble_gatt_svc_def gatt_svcs[] = {
    {
        .type = BLE_GATT_SVC_TYPE_PRIMARY,
        .uuid = &vib_svc_uuid.u,
        .characteristics = (struct ble_gatt_chr_def[]) {
            {
                .uuid = &vib_chr_uuid.u,
                .access_cb = vib_access_cb,           // called when the phone reads
                .flags = BLE_GATT_CHR_F_READ | BLE_GATT_CHR_F_NOTIFY,
                .val_handle = &vib_val_handle,       // you notify against this handle
            },
            { 0 } // terminator
        },
    },
    { 0 }
};
```

To push a new reading to a subscribed phone, wrap the float and notify:

```
static void notify_rms(uint16_t conn_handle, float rms) {
    struct os_mbuf *om = ble_hs_mbuf_from_flat(&rms, sizeof(rms));
    ble_gatts_notify_custom(conn_handle, vib_val_handle, om); // only if it subscribed
}
```

14.5 Code walkthrough, line by line

Two blocks. The first declares your GATT table; the second pushes a value. The stack initialisation and advertising setup are long and largely boilerplate, so start from the `bleprph` example in your own ESP-IDF installation rather than typing them — but these two pieces are genuinely yours.

Block 1, line 1 · `static const struct ble_gatt_svc_def gatt_svcs[] = {`

An array of service definitions — a **declarative table** rather than a sequence of function calls. You describe the shape of your data and hand the whole thing to the stack, which walks it and exposes everything.

Three qualifiers, all load-bearing. `static` keeps it private to this file. `const` lets the compiler place it in flash rather than RAM, which matters because it is never modified. And it is at **file scope** for a reason that catches people: the stack keeps a pointer to this table and reads it long after registration returns. A table declared inside a function would go out of scope and the stack would be walking freed stack memory — a crash that appears much later than its cause.

Block 1, line 3 · `.type = BLE_GATT_SVC_TYPE_PRIMARY`

Declares this a **primary** service — a top-level entry a client will find during discovery. The alternative, `SECONDARY`, is for a service meant to be included inside another rather than discovered on its own, and it is rare.

Block 1, line 4 · `.uuid = &vib_svc_uuid.u`

The service's 128-bit identifier. The `.u` is a small NimBLE idiom worth decoding: UUIDs come in 16-, 32- and 128-bit forms, so NimBLE defines a union with a common header, and `.u` selects that generic header member. Taking its address gives the stack a pointer it can inspect to learn which size it is dealing with.

Block 1, line 5 · `.characteristics = (struct ble_gatt_chr_def[]) { ... }`

A **compound literal**, and this syntax is the part of the listing most likely to look alien. The form `(type[]){ ... }` constructs an anonymous array right there inside the initialiser, without declaring a separate named variable for it.

It is a C99 feature and it appears constantly in this style of declarative table, so it is worth recognising rather than being puzzled by. The equivalent without it would be declaring `static const struct ble_gatt_chr_def vib_chrs[] = {...}` above and then writing `.characteristics = vib_chrs` — identical result, more names to invent.

One thing to be careful about: a compound literal at **file scope** has static storage duration, which is what makes this safe. The same syntax written *inside a function* would have automatic storage and would be destroyed on return — the same lifetime trap as the table itself.

Block 1, line 8 · `.access_cb = vib_access_cb`

The function the stack calls when a client **reads** this characteristic. Note this is a callback — you do not send the value, you supply a function that produces it on demand. That is the pull half of §14.3, and it is why a read always returns fresh data rather than something you pushed earlier.

The same callback also handles **writes** if the characteristic permits them; it receives an argument telling it which operation is happening.

Block 1, line 9 · `.flags = BLE_GATT_CHR_F_READ | BLE_GATT_CHR_F_NOTIFY`

Two independent option bits combined with a **bitwise OR**. Each flag is a distinct bit in the field, so ORing sets both — the standard C idiom for "select several options at once," the same mechanism as the pin masks in Lab 1.

These flags are not merely advisory. They are **advertised to the client** during discovery, so a phone app inspecting your device sees exactly these properties listed and will grey out operations you did not permit. Declaring `NOTIFY` is also what makes the stack create the CCCD descriptor that a client needs in order to subscribe — omit the flag and there is nothing to subscribe to.

Other flags you will meet: `F_WRITE`, `F_WRITE_NO_RSP` (fire and forget, faster), `F_INDICATE` (acknowledged notify), and the encryption-required variants.

Block 1, line 10 · `.val_handle = &vib_val_handle`

An **output parameter** inside a declarative table, which is slightly unusual. You supply the address of a `uint16_t`, and during registration the stack writes into it the numeric handle it assigned to this characteristic.

You need that number later because `ble_gatts_notify_custom` identifies the characteristic by handle, not by UUID — handles are small integers and cheap to compare, whereas matching a 128-bit UUID on every notification would be wasteful. If you forget this field, the handle stays zero and your notifications go nowhere.

Block 1, lines 12 and 15 · `{ 0 }`

Sentinel terminators, and there is one at each level of the tree: line 12 ends the characteristics array, line 15 ends the services array.

These arrays carry no length field, so the stack walks them until it hits a zeroed entry — exactly like the `\0` that ends a C string, or the `NULL` that ends `argv`. Recognising the sentinel-terminated-array pattern is broadly useful; it appears all over C APIs.

Omit a terminator and the stack reads off the end of your array into whatever memory follows, interpreting it as more services. The result is a crash or a device advertising nonsense, and because the damage is proportional to whatever garbage happened to be adjacent, the symptom varies run to run — which makes it genuinely unpleasant to debug.

Block 2, line 1 · `static void notify_rms(uint16_t conn_handle, float rms)`

Takes the **connection handle** identifying which client to notify. That parameter exists because a peripheral can hold several simultaneous connections, and a notification goes to one specific client — there is no broadcast-to-all-subscribers call.

You obtain this handle from the connect event and store it; a common bug is notifying against a stale handle after the client has disconnected.

Block 2, line 2 · `struct os_mbuf *om = ble_hs_mbuf_from_flat(&rms, sizeof(rms));`

BLE data travels through the stack in **mbufs** — chained buffers borrowed from network-stack design. The point of a chained buffer is that a packet can gain and lose headers as it moves down the layers without anyone copying the payload; each layer just links its header onto the front.

Your `float` is an ordinary variable in contiguous memory — "flat" — so this call copies it into a buffer from the stack's pool. Because it **copies**, your local `rms` can go out of scope immediately afterwards and nothing breaks.

Worth knowing: mbufs come from a fixed-size pool, so this can return `NULL` under memory pressure. This listing does not check, and production code should.

Block 2, line 3 · `ble_gatts_notify_custom(conn_handle, vib_val_handle, om);`

Sends the notification to that client, against that characteristic handle. The stack takes ownership of the mbuf and frees it, so you must not touch or free `om` afterwards.

The comment is the caveat from §14.3: **this only reaches the phone if it subscribed**. The call succeeds either way and returns no indication that nobody was listening.

NOTE

You are sending four raw bytes, and nothing says so

This transmits a float in the chip's native binary representation — four bytes of IEEE-754, little-endian. BLE carries **no type information whatsoever**; a characteristic value is just a byte array, and both ends must independently agree what it means.

So a phone app must know to reinterpret those four bytes as a float, which is why nRF Connect will show you something meaningless unless you tell it the format. Real profiles either use a documented standard format — the SIG defines exact encodings for adopted characteristics — or send text, which is self-describing at the cost of size.

Being aware you have made an undocumented assumption is the mark of someone who has thought about it. §14.7 has you try the text version and compare.

14.6 Docs & references

DOCS

Trace every line to its source

NimBLE in ESP-IDF — API Reference → Bluetooth → **NimBLE-based Host APIs**. Dense, but it is the authoritative reference for `ble_gatt_svc_def` and friends.

The bleprph example — in your own installation under `examples/bluetooth/nimble/bleprph`. This is the best starting point by a wide margin: it is complete, it compiles against your exact IDF version, and it shows the advertising and connection handling this listing omits.

GATT concepts — the Bluetooth SIG's own introduction to GATT, and Nordic's developer documentation, which is unusually clear on services, characteristics and the CCCD.

Assigned numbers — the SIG publishes the standard 16-bit UUIDs. Worth a look so you know when to reuse a standard service rather than invent one.

14.7 Verify, troubleshoot, your turn

Verify. Open nRF Connect and scan. Your device appears by name. Connect, and you see your service listed with its characteristic. Tap read and you get the current value; tap subscribe and values start arriving on their own as you shake the board. That live stream on a phone is the most demonstrable thing in this entire series — it works at a career fair, in a hallway, with no laptop.

Troubleshoot

Symptom	Cause and fix
Device never appears in a scan	Advertising was not started, or NimBLE is not enabled in <code>menuconfig</code> .
Connects, but no services shown	The GATT table was not registered, or a <code>{ 0 }</code> terminator is missing.

Symptom	Cause and fix
Read works, notifications never arrive	The client has not subscribed. Tap the subscribe icon.
Value looks like nonsense on the phone	The app is interpreting four raw float bytes as something else. Ask it to show hex and check.
Disconnects after a few seconds	Connection parameters, or the peripheral is not responding to the connection event in time.

Your turn

1. **Notify from the Lab 7 feature task** so the phone shows live vibration. Then tap the board and watch the number move on your screen — that is a complete product demo in miniature.
2. **Add a writable characteristic** that lets the phone change a threshold on the device. Now data flows both ways and you have a configurable product rather than a broadcaster.
3. **Send the value as text** instead of raw bytes and compare how the two look in nRF Connect. That comparison makes the encoding trade-off concrete: readability versus size, the same choice as JSON versus binary in Lab 13.

The series at a glance

Each lab is a complete, independently buildable ESP-IDF project. This is Lab 14.

#	Lab	Phase
00	Toolchain: hello, ESP32-S3	Foundations
01	GPIO: RGB LED and BOOT button	Foundations
02	FreeRTOS tasks and drift-free timing	Foundations
03	UART and the loopback test	Foundations
04	Hardware timers and the 1 kHz problem	Foundations
05	I ² C and the IMU bring-up	Foundations
06	PWM with the LEDC peripheral	Foundations
07	The sensor pipeline (capstone)	Foundations
08	GPIO interrupts and ISR-safe code	Past the coursework
09	SPI, full-duplex and fast	Past the coursework
10	NVS: data that survives power loss	Past the coursework
11	Unit testing with Unity	Past the coursework
12	Deep sleep and waking up	Past the coursework
13	Wi-Fi station and JSON telemetry	Past the coursework
14 ←	BLE GATT with NimBLE	Past the coursework
15	OTA and the rollback safety net	Past the coursework
16	Priority inversion and mutexes	Past the coursework
17	CAN (TWAI): broadcast by message ID	Past the coursework
18	PID, anti-windup and derivative kick	Past the coursework
19	The ML inference harness	Gap-closers
20	Continuous ADC: DMA-backed acquisition	Gap-closers
21	On-chip debugging: JTAG, OpenOCD, GDB	Gap-closers
22	A real display (SPI TFT + DMA)	Gap-closers
23	Production test mode	Gap-closers

Written against ESP-IDF v5.5 on an ESP32-S3 N16R8 module. Pin assignments are choices, not constraints — the ESP32-S3 GPIO matrix routes almost any peripheral to almost any pin, with the exception of GPIO26–GPIO37, which are committed to the on-board flash and PSRAM and must never be used.